

Алгоритмы и структуры данных
Разбор задачи №407
Trapping Rain Water II
Беспалов Максим Константинович
Группа Б9124-02.03.01СЦТ
17 февраля 2026 г.

1. Постановка задачи

Дана прямоугольная сетка размера $m \times n$, где каждая ячейка содержит целое неотрицательное число — высоту этой ячейки над уровнем моря. После дождя вода может заполнять низины, если они окружены более высокими клетками. Требуется найти общий объём воды, который может быть захвачен такой рельефной картой.

Формально:

Дан массив `heightMap[m][n]`. Вода может попасть в ячейку, если все пути от неё к границе сетки проходят через клетки с большей или равной высотой (при этом вода не переливается через край, если на пути встречается более низкая клетка — это создаёт “чашу”). Объём воды в каждой ячейке равен разности между уровнем воды в этой чаше и собственной высотой ячейки (если эта разность положительна). Суммарный объём — искомая величина.

1.1 Ограничения

- $1 \leq m, n \leq 200$
- $0 \leq \text{heightMap}[i][j] \leq 2 \times 10^4$

1.2 Примеры

Пример 1

Вход:

[[1,4,3,1,3,2],

[3,2,1,3,2,4],

[2,3,3,2,3,1]]

Выход: 4

Пояснение: образуются две небольшие лужицы объёмом 1 и 3, итого 4.

Пример 2

Вход:

[[3,3,3,3,3],

[3,2,2,2,3],

[3,2,1,2,3],

[3,2,2,2,3],

[3,3,3,3,3]]

Выход: 10

вода заполняет внутреннюю область глубиной 1 (центральная клетка) и 2 (соседние кольца).

2. Анализ задачи

Задача является двумерным обобщением классической задачи “Trapping Rain Water” (одномерный случай). В одномерном варианте достаточно двух указателей и отслеживания максимальных высот слева и справа. В двумерном случае вода может просачиваться во все стороны, поэтому требуется более сложный подход.

Основная идея: вода не может вытекать через границу сетки, значит, минимальная высота на границе определяет максимальный уровень воды, который способна удержать внутренняя область. Если из границы в какую-то внутреннюю ячейку ведёт путь, все клетки которого не выше некоторого порога, то эта ячейка будет заполнена водой до уровня минимальной граничной высоты на этом пути.

Это наблюдение приводит к алгоритму, напоминающему алгоритм Дейкстры: мы начинаем с граничных клеток и постепенно “поднимаем уровень воды”, обрабатывая клетки в порядке возрастания их текущей граничной высоты.

3. Алгоритм решения

Введём приоритетную очередь (min-heap), которая будет хранить тройки (высота, x , y). Начальное состояние: все граничные клетки помещаются в очередь и помечаются посещёнными. Затем многократно извлекаем элемент с наименьшей высотой из очереди и рассматриваем его четырёх соседей. Если сосед ещё не посещён:

- Если высота соседа меньше текущей высоты h , то вода может заполнить эту ячейку до уровня h , и мы добавляем к ответу разность $h - \text{heightMap}[x][y]$.
- Помечаем соседа посещённым и добавляем его в очередь с новой высотой $\max(\text{heightMap}[x][y], h)$. Эта новая высота отражает тот факт, что теперь граница в этом направлении поднялась до максимума из исходной высоты ячейки и уровня воды, который мы только что рассматривали.

Процесс продолжается, пока очередь не опустеет. Ответ накопится в переменной `ans`.

Обоснование:

Граничные клетки — это единственный путь, через который вода может покинуть область. Обработывая клетки в порядке возрастания их “граничной высоты”, мы гарантируем, что когда мы рассматриваем ячейку, все возможные пути с меньшей высотой уже обработаны, и мы знаем максимальный уровень воды, который может быть удержан над этой ячейкой. Если соседняя ячейка ниже текущей границы, она будет заполнена водой до этой границы; если выше — она сама становится новой границей для дальнейшего распространения.

4. Реализация

Реализация алгоритма на C++ с использованием `priority_queue`.

```
1  class Solution {
2  public:
3      int trapRainWater(vector<vector<int>>& heightMap) {
4          using tii = tuple<int, int, int>;
5          priority_queue<tii, vector<tii>, greater<tii>> pq;
6          int m = heightMap.size(), n = heightMap[0].size();
7          bool vis[m][n];
8          memset(vis, 0, sizeof vis);
9          for (int i = 0; i < m; ++i) {
10             for (int j = 0; j < n; ++j) {
11                 if (i == 0 || i == m - 1 || j == 0 || j == n - 1) {
12                     pq.emplace(heightMap[i][j], i, j);
13                     vis[i][j] = true;
14                 }
15             }
16         }
17         int ans = 0;
18         int dirs[5] = {-1, 0, 1, 0, -1};
19         while (!pq.empty()) {
20             auto [h, i, j] = pq.top();
21             pq.pop();
22             for (int k = 0; k < 4; ++k) {
23                 int x = i + dirs[k], y = j + dirs[k + 1];
24                 if (x >= 0 && x < m && y >= 0 && y < n && !vis[x][y]) {
25                     ans += max(0, h - heightMap[x][y]);
26                     vis[x][y] = true;
27                     pq.emplace(max(heightMap[x][y], h), x, y);
28                 }
29             }
30         }
31         return ans;
32     }
```

Saved

Ln 1, Co

5. Пример пошагового выполнения

Рассмотрим входные данные из первого примера:

$$\text{heightMap} = \begin{bmatrix} 1 & 4 & 3 & 1 & 3 & 2 \\ 3 & 2 & 1 & 3 & 2 & 4 \\ 2 & 3 & 3 & 2 & 3 & 1 \end{bmatrix}$$

Размеры: $m = 3$, $n = 6$.

Шаг 0. Инициализация. Все граничные клетки помещаются в min-heap с их высотами. Отметим их как посещённые.

Граничные клетки (координаты и высота):

(0,0):1; (0,1):4; (0,2):3; (0,3):1; (0,4):3; (0,5):2;

(1,0):3; (1,5):4;

(2,0):2; (2,1):3; (2,2):3; (2,3):2; (2,4):3; (2,5):1.

Шаг 1. Извлекаем минимальный элемент: высота 1, координаты (0,0). Проверяем соседей:

- (1,0): высота $3 \geq 1 \rightarrow$ вода не добавляется, помечаем посещённым и кладём в кучу с высотой $\max(3,1)=3$.
- (0,1): высота $4 \geq 1 \rightarrow$ добавляем с высотой 4.

Шаг 2. Извлекаем следующий минимум: опять высота 1, теперь (0,3). Соседи:

- (1,3): высота $3 \geq 1 \rightarrow$ добавляем с высотой 3.
- (0,2): высота $3 \geq 1 \rightarrow$ добавляем с высотой 3.
- (0,4): высота $3 \geq 1 \rightarrow$ добавляем с высотой 3.
(Сосед слева (0,2) ещё не посещён; сосед справа (0,4) тоже.)

Шаг 3. Извлекаем высоту 1 (2,5). Соседи:

- (1,5): высота $4 \geq 1 \rightarrow$ добавляем с высотой 4.
- (2,4): высота $3 \geq 1 \rightarrow$ добавляем с высотой 3.

Шаг 4. Извлекаем высоту 2 (0,5). Соседи:

- (1,5) уже посещён; (0,4) уже посещён.

Шаг 5. Извлекаем высоту 2 (2,0). Соседи:

- (1,0) уже посещён; (2,1) высота $3 \geq 2 \rightarrow$ добавляем с высотой 3.

Шаг 6. Извлекаем высоту 2 (2,3). Соседи:

- (1,3) уже посещён; (2,2) высота $3 \geq 2 \rightarrow$ добавляем с высотой 3; (2,4) уже посещён.

... процесс продолжается, пока куча не опустеет.

В итоге при обработке клетки (1,2) с высотой 1, когда она будет извлечена, её соседи уже будут иметь граничную высоту не ниже 2 или 3, и разница добавится к ответу. Подсчёт даёт суммарный объём 4.

6. Анализ сложности

- **Временная сложность:**

Каждая из $m \times n$ клеток попадает в приоритетную очередь ровно один раз. Операции вставки и извлечения из кучи занимают $O(\log(mn))$. Следовательно, общее время работы

$$O(m \cdot n \cdot \log(m \cdot n)).$$

- **Пространственная сложность:**

Необходимо хранить очередь, которая в худшем случае содержит все $m \times n$ элементов, и булевый массив `vis` размером $m \times n$. Дополнительная память:

$$O(m \cdot n).$$

При максимальных размерах $200 \times 200 = 40\,000$ ячеек алгоритм выполняется быстро и укладывается в ограничения.

7. Затраты по памяти

- Массив `vis`: $m \times n$ байт (если `bool` занимает 1 байт) — до 40 КБ.
- Приоритетная очередь: хранит до $m \times n$ записей, каждая запись содержит высоту (`int`) и две координаты (`int`) — примерно 3×4 байта = 12 байт на элемент, итого до 480 КБ.
- Прочие переменные: несколько целых.

Общая память не превышает 1 МБ, что является константным для данной размерности.

8. Ключевые особенности решения

- **Моделирование подъёма уровня воды** с помощью `min-heap` — классический приём для задач типа “водосбор”.
- **Аналогия с алгоритмом Дейкстры**: мы ищем “кратчайший путь” от границы до каждой клетки, где стоимость пути — максимальная высота на пути. Тогда уровень воды в клетке равен минимальной из таких максимальных высот по всем путям от границы.
- **Обработка только непосещённых соседей** предотвращает повторное посещение и гарантирует, что каждая клетка рассматривается один раз.

- **Обновление высоты при добавлении в кучу** как максимум исходной высоты и текущей границы обеспечивает корректное распространение “фронта воды”.
-

9. Заключение

Представленный алгоритм эффективно решает задачу Trapping Rain Water II, используя приоритетную очередь для имитации подъёма уровня воды от наименьших граничных высот. Благодаря этому подходу достигается временная сложность $O(mn \log(mn))$, что достаточно быстро при максимальных размерах входных данных. Решение проходит все тесты платформы LeetCode и является оптимальным для данной задачи.